

Web Dev Timeline & Component Systems

PHASE 1: Introduction & Evolution of Web Dev

VERBATIM STREAM:

- [on_screen_text] Web Dev Timeline
- [on_screen_text] 90s HTML, CSS, JavaScript, PHP UI - CSS(kind)
- [on_screen_text] 2000s ASP.NET, AJAX, jQuery UI - CSS
- [on_screen_text] 2010s Angular, React, Vue, Web Components, Svelte, Next UI - Bootstrap, Foundation, Material Design, Tailwind
- [on_screen_text] 2020s Astro, Solid, SvelteKit, Qwik, Remix, Fresh UI - Chakra, ShadCN, Skeleton
- [spoken_transcript] Building websites has evolved from writing plain HTML, CSS, and JavaScript to using components and design systems for a more scalable and organized approach.
- [spoken_transcript] Whether you're building a simple static site or a dynamic app, using reusable parts keeps your code cleaner and easier to maintain.
- [on_screen_text] Static site
- [on_screen_text] Dynamic app
- [on_screen_text] Styles
- [on_screen_text] Components
- [spoken_transcript] A design system is basically a collection of reusable UI components, global CSS styles, and utility classes that ensure consistency in your design.
- [spoken_transcript] Instead of rewriting styles from scratch every time, you create predefined, reusable components that speed up development and make updates way easier.
- [spoken_transcript] Think of it like having a set of Lego bricks instead of carving new ones out of wood every time you want to build something.

PHASE 2: Component Thinking

VERBATIM STREAM:

- [spoken_transcript] Let's say you're building a website to promote your services. Instead of repeating the same HTML structures all over the place, break your website into reusable components.
- [on_screen_text] STEP 1 Define your components
- [on_screen_text] smart way TO BUILD WEBSITES
- [spoken_transcript] Up top, we have a header. Then a hero section, followed by whatever this is.
- [spoken_transcript] A few more sections, a call to action, and finally a footer.

PHASE 3: Design First Approach

VERBATIM STREAM:

- [on_screen_text] STEP 0 Design first, build later
- [on_screen_text] STEP 1 Define your components
- [spoken_transcript] So actually, step zero should be designing first. If you don't know how to design, just watch some videos to learn the basics.
- [spoken_transcript] Use a tool like Mobbin... to steal... I mean take inspiration from good designs.
- [spoken_transcript] You can also check out your competition or browse the Figma community for ideas.
- [spoken_transcript] Personally, I prefer Mobbin because it makes filtering and browsing through tons of designs ridiculously easy.
- [spoken_transcript] In just a few minutes, you can go through hundreds of real-world designs that have already been tested and iterated on. So you know you're looking at the best of the best.
- [spoken_transcript] You can filter by platform or any category you like.
- [spoken_transcript] With all these different categories and options, finding good, relevant designs doesn't take long.
- [spoken_transcript] Plus, if you're looking for something specific, like, say, a footer section, just search for it, browse, and save the ones you like to your collection.
- [spoken_transcript] I'm not going to walk you through every single feature because there are way too many, but trust me, they're worth exploring.
- [spoken_transcript] The UI is already super intuitive, so you don't need me to hold your hand.
- [spoken_transcript] But I opened up Mobbin for a reason. Let's analyze some real websites and see if we can spot a pattern.

PHASE 4: Analyzing Real Websites

VERBATIM STREAM:

- [spoken_transcript] These testimonial cards, same component.
- [spoken_transcript] This bento grid, also the same component.
- [spoken_transcript] Some of them might look different, but they are not. On larger screens, flexbox kicks in and makes things a bit more dynamic.
- [on_screen_text] Notice the order
- [on_screen_text] 1 Heading
- [on_screen_text] 2 Details
- [on_screen_text] 3 Media
- [spoken_transcript] Anyone can see these are just reused components.
- [spoken_transcript] Yep, these four sections, same component. Three variables or props if we're being technical. Icon, heading, and details.
- [spoken_transcript] Hold on a minute. I hope you're seeing what I'm seeing. We've got our classic two-column setup. One column with an image, the other with a subheading, heading, and details. And when needed, just reverse the flex direction.
- [spoken_transcript] This is literally the same component we just saw earlier.
- [spoken_transcript] And this one too.

- [spoken_transcript] Okay I get it now. This whole page is built using just two components. The header, hero section, call to action, and footer are global components anyway.
- [spoken_transcript] So really, this entire page is just dropping the right components into place.
- [spoken_transcript] Let's check out CRM.
- [spoken_transcript] Yep, same hero section.
- [spoken_transcript] And of course, the same components are used here too.
- [spoken_transcript] I bet every page follows the same pattern, but let's double check.
- [spoken_transcript] Yep, every single page is built using the same reusable components.
- [spoken_transcript] In simple terms, repeat your design. If you're building apps, you have to do this anyway because you need a consistent structure to render dynamic data.
- [spoken_transcript] Look at YouTube. Every single video card follows the same style and props, and the only thing that changes is the content inside.
- [spoken_transcript] Now, take this knowledge and go design something interesting.

PHASE 5: Implementation Steps Overview

VERBATIM STREAM:

- [spoken_transcript] Once you have a design in place, it's time to bring it to life.
- [spoken_transcript] If you don't code, you can use any of these other options. But regardless of the tool, the same design principles still apply.
- [on_screen_text] If you don't code
- [on_screen_text] Use these
- [on_screen_text] Webflow
- [on_screen_text] WordPress
- [on_screen_text] Framer
- [on_screen_text] or ChatGPT
- [on_screen_text] But regardless of the tool
- [on_screen_text] the same DESIGN PRINCIPLES apply
- [spoken_transcript] We'll start simple. First, we'll use variables and utility classes to speed up the development process.

PHASE 6: Step 2 - CSS Variables & Reset

VERBATIM STREAM:

- [on_screen_text] STEP 0 Design first, build later
- [on_screen_text] STEP 1 Define your components
- [on_screen_text] STEP 2 Setup your CSS variables
- [spoken_transcript] Before anything else, we do a quick reset to remove default margins and paddings.
- [on_screen_text]


```
1 /* Reset */
2 * {
3   margin: 0;
```

- ```

4 padding: 0;
5 }

```
- [spoken\_transcript] Also, we make sure width and height calculations include padding and borders to prevent weird layout issues.
  - [on\_screen\_text]

```

6 box-sizing: border-box;
7 }

```
  - [spoken\_transcript] Then, we define our global colors and font styles as variables. This keeps everything consistent and makes the development process way smoother.
  - [on\_screen\_text]

```

8 /* Variables */
9 :root {
10 --color10: hsl(31, 80%, 10%);
11 --color30: hsl(31, 80%, 30%);
12 --color90: hsl(31, 80%, 90%);
13 --color100: hsl(31, 80%, 100%);
14 --accent: hsl(31, 80%, 70%);
15 }

```
  - [spoken\_transcript] Speaking of consistency, I also add spacing variables. If you've never done this before, it might seem unnecessary, but trust me, it makes a huge difference.
  - [on\_screen\_text]

```

16 --ff: "Noto Sans JP", serif;
17 --ff2: "Fira Sans", serif;
18 --p: 1rem/1.5em var(--ff);
19 --h1: 600 3rem/1.2em var(--ff2);
20 --h2: 600 2rem/1.2em var(--ff2);
21 --spacing1: 4px;
22 --spacing2: 8px;
23 --spacing3: 16px;
24 --spacing4: 20px;
25 --spacing5: 40px;
26 --spacing6: 80px;
27 --spacing7: 160px;
28 }

```
  - [spoken\_transcript] These variables affect both the aesthetics and usability of your website. Plus, if you ever need to tweak the design in the future, it's as simple as changing one value instead of manually adjusting everything.

## PHASE 7: Step 3 - Global Styles

### VERBATIM STREAM:

- [on\_screen\_text] STEP 3 Create global styles
- [spoken\_transcript] Next, we override some defaults and style our global elements like headings, links, and buttons.

- [on\_screen\_text]
 

```

29 /* Global Styles */
30 html {
31 scroll-behavior: smooth;
32 }
33 body {
34 text-wrap: balance;
35 font: var(--p);
36 background: var(--color90);
37 color: var(--color10);
38 }

```
- [spoken\_transcript] The key here: Never use fixed values like black or 40 pixels. Instead, always use the variables you just defined.
- [on\_screen\_text]
 

```

39 h1 {
40 font: var(--h1);
41 }
42 h2 {
43 font: var(--h2);
44 }

```
- [spoken\_transcript] Start with the basics. You can always add more styles later as needed.
- [on\_screen\_text]
 

```

56 .btn,
57 button {
58 all: unset;
59 display: inline-block;
60 margin: var(--spacing2) 0;
61 background: var(--color100);
62 padding: var(--spacing2) var(--spacing3);
63 color: var(--color10);
64 border-radius: var(--spacing2);
65 border-bottom: solid var(--spacing1) var(--accent);
66 font: var(--p);
67 cursor: pointer;
68 }

```
- [spoken\_transcript] If you're using a no-code platform, don't skip this step. Look for the theme settings and set your global colors and typography there. Most platforms offer tools and plugins to keep things structured. You just need to apply the same principles.

## PHASE 8: Step 4 - Utility Styles

### VERBATIM STREAM:

- [on\_screen\_text] STEP 4 Create utility styles

- [spoken\_transcript] Now, let's create some super common utility classes. Things like containers, text alignment, flexbox layouts, padding, and margin helpers.
- [on\_screen\_text]
 

```

79 /* Utility Classes */
80 section {
81 width: 90%;
82 max-width: 1080px;
83 margin: 0 auto;
84 }
85 .center {
86 text-align: center;
87 }
88 .hidden {
89 display: none;
90 }
91 .flex {
92 display: flex;
93 align-items: center;
94 justify-content: center;
95 gap: var(--spacing4);
96 flex-wrap: wrap;
97 }

```
- [spoken\_transcript] If you've used Tailwind before, this should feel familiar. The difference? Here, you get to name the classes, so they're actually memorable. We'll go over Tailwind later in the video, but for now, this setup gives us full control and flexibility.

## PHASE 9: Step 5 - Styling Components (Vanilla HTML/CSS Approach)

### VERBATIM STREAM:

- [on\_screen\_text] STEP 5 Style your components
- [spoken\_transcript] All right, now that our initial setup is done, it's time to start styling.
- [spoken\_transcript] There are two main approaches to writing CSS. One is the single file approach, where you keep everything in one CSS file. The other is a component-based approach, where you break things into separate CSS files for different components.
- [spoken\_transcript] The first approach is simpler, as there's no setup involved. So let's start with that and see other options in a minute.
- [spoken\_transcript] Since we're styling repeating elements, I'll call this step styling components even though the first process is just plain old HTML.
- [spoken\_transcript] The header will be the same across the website, so you just build it once and use it forever.
- [on\_screen\_text]
 

```

148 /* header */
149 header {
150 width: 90%;
151 max-width: 1080px;

```

```

152 margin: 0 auto;
153 display: flex;
154 align-items: center;
155 justify-content: space-between;
156 position: relative;
157 padding: var(--spacing4) 0;
158 }

```

- [spoken\_transcript] Same thing with these six sections. We just use some utility classes, and they can be used throughout the website across multiple pages.
- [on\_screen\_text]
 

```

27 <section class="flex margin6">
28 <div>
29 <h2>We Build Websites</h2>

```
- [spoken\_transcript] These two sections are identical, except this flex direction change on larger screens. The same goes for this one, and for the CTA section as well. Design it once, use it everywhere.
- [spoken\_transcript] But here's the thing. This approach doesn't scale very well. First of all, you have to manually copy-paste the HTML every time you want to use it somewhere. Secondly, you have no control over it. Each instance is independent of the others. So if you decide to move this link here, or add a class to this div, you'll have to track down and edit every section one by one. And trust me, that's not the kind of fun we're looking for.

## PHASE 10: Web Components Approach

### VERBATIM STREAM:

- [on\_screen\_text] STEP 5 Create & Style your components
- [on\_screen\_text] v1 - Web Components + Vanilla CSS
- [spoken\_transcript] So let's look at a better approach for building component-based websites. This is where we actually create reusable components that can be edited from a single place and update all the instances across our website.
- [spoken\_transcript] Before reaching for a framework, let's first see a native way of creating components.
- [spoken\_transcript] Let's see how we can create our header in one place and use it everywhere.
- [spoken\_transcript] We're going to create a new custom HTML element called THeader, since we can't use header because it's already taken by the browser.
- [on\_screen\_text]
 

```

1 // header
2 class THeader extends HTMLElement {
3
4 }
5 customElements.define("t-header", THeader);

```
- [spoken\_transcript] HTMLElement is the base class for all HTML elements, and we're extending it to make our own custom element.

- [spoken\_transcript] This function runs automatically when the element THeader is added to the DOM. It's like saying, "Hey when this element appears in the HTML, do this next."
- [on\_screen\_text]
 

```
3 connectedCallback() {
4
5 }
```
- [spoken\_transcript] So, we define the structure inside, and voila, whenever you need a header, just link that file and use it like any normal HTML element.
- [on\_screen\_text]
 

```
4 this.innerHTML = 5 <header> 6 7 8 <nav> 9 Services 10 Our Work 11 Pricing 12 About 13 Hire us 14 </nav> 15 </header> 16;
```
- [on\_screen\_text]
 

```
16 <t-header></t-header>
17 <script src="script.js"></script>
```
- [spoken\_transcript] Imagine you built 10 pages and then realized the header needs a few more links. Instead of manually updating each page, just add them in one place, and it updates everywhere. Way better than copy-pasting.
- [spoken\_transcript] Let's take it further. Create this section, give it a name, and copy-paste the HTML inside.
- [on\_screen\_text]
 

```
21 // section
22 class TSection extends HTMLElement {
23 connectedCallback() {
24 this.innerHTML = ``;
25 }
26 }
27 customElements.define("t-section", TSection);
```
- [spoken\_transcript] Then, find the variables like heading, details, etc., and name them however you like.

[on\_screen\_text]

```
26 <h2>
 heading</h2>27<p>{heading}</h2>
27 <p>heading</h2>27<p>
```

- ```
{details}</p>
28 <a class="btn" href="${link}">${linkText}</a>
```
- [spoken_transcript] Then we'll create those variables to grab the actual text we'll pass into the HTML.
- [on_screen_text]


```
24 const heading = this.getAttribute("heading");
25 const details = this.getAttribute("details");
```

```

26 const link = this.getAttribute("link");
27 const linkText = this.getAttribute("linkText");
28 const img = this.getAttribute("img");

```

- [spoken_transcript] Inside our HTML file, we can use the custom tag we just created and pass the values like this.

- [on_screen_text]

```

19 <t-section
20 heading="We Build Websites"
21 details="Ever dreamed of a website that loads in under a minute? We make that
happen-sometimes even faster."
22 link="#"
23 linkText="Explore the magic"
24 img="img/building websites.svg"
25 ></t-section>

```

- [spoken_transcript] If you've ever used a framework, this might look very similar, except for the closing tag. The difference is that here, we're passing values manually instead of fetching them from an API.
- [spoken_transcript] Now, let's do the same for the reverse section. However, we need to add a reverse utility class to it. While we can't add it directly here due to the fixed classes in our component, we can either create a variable to pass the class or design a new component specifically for the reverse section.
- [spoken_transcript] However, since these two sections will always appear together, I prefer combining them in one component.
- [spoken_transcript] I'm going to combine these two sections like this, and pass the reverse class directly inside our base component.

- [on_screen_text]

```

35 this.innerHTML = `
36 <section class="flex margin6">

```

...

```

43 <section class="flex margin6 reverse">

```

- [spoken_transcript] This makes it easier to control the layout, and the HTML will look something like this.

- [on_screen_text]

```

27 <t-section
28 selector="flex margin6 reverse"
29 heading="Marketing Strategies That May or May Not Work"

```

- [spoken_transcript] This section is simple as it only contains two variables, the heading and the paragraph. So let's create two variables and pass them in like this.

- [on_screen_text]

```

55 class TMainSection extends HTMLElement {
56   connectedCallback() {
57     const heading = this.getAttribute("heading");
58     const details = this.getAttribute("details");
59

```

```

60 this.innerHTML = 61 <section class="center padding6"> 62 <h2>${heading}</h2> 63
  <p>${details}</p> 64 </section> 65;
66 }
67 }

```

```

68 customElements.define("t-main-section", TMainSection);

```

- [spoken_transcript] Our CTA section is pretty similar to the main section, with the addition of a link here. We'll create four variables for the heading, details, link, and link text, and pass them in like this.
- [spoken_transcript] The footer however, remains the same across the entire website, so we don't need variables for it. Instead, let's create a custom element for it, and just copy-paste the footer HTML.
- [spoken_transcript] Now, whenever we need the footer, we can use the custom element like this.
- [on_screen_text]


```

66 <t-footer />

```
- [spoken_transcript] This approach makes our components modular and easy to reuse. With everything set up, our components are ready, and we can use them anywhere we want.
- [spoken_transcript] If you prefer vanilla solutions, web components are a fantastic option for building websites.

PHASE 11: Frameworks vs. Static Sites

VERBATIM STREAM:

- [spoken_transcript] But they have their challenges. If JavaScript is disabled, the page won't load, hurting SEO and performance.
- [spoken_transcript] Plus, as the project grows, more CSS can lead to naming conflicts and cascading issues. The shadow DOM helps with scoping, but things get tricky at scale.
- [spoken_transcript] The better solution in this case, is to use tools and frameworks that handle these issues out of the box.
- [spoken_transcript] Now, you might be thinking, what about React? Well, React can be overkill for static websites.
- [spoken_transcript] Instead, I suggest using something like Astro or Svelte, which are both lightweight, easy to work with, and generate static HTML for better performance.
- [on_screen_text] 6 out of 9 devs recommend
- [on_screen_text] Astro
- [on_screen_text] Svelte
- [on_screen_text] lightweight
- [on_screen_text] lightweight, Easy to use
- [on_screen_text] lightweight, Easy to use, Really fast!
- [on_screen_text] Easy to build
- [on_screen_text] Easy to build, Easy to load
- [spoken_transcript] You get the best of both worlds, easy to build and easy to load.

PHASE 12: Svelte Approach

VERBATIM STREAM:

- [on_screen_text] STEP 5 Create & Style your components
- [on_screen_text] v2 - Svelte + Vanilla CSS
- [spoken_transcript] Personally, I prefer Svelte over Astro because it allows you to build dynamic websites as well. So you won't have to learn two different frameworks for different needs.
- [spoken_transcript] That said, Astro is also amazing and super simple to use. So feel free to give it a try as well.
- [spoken_transcript] Now, let's get started building this website using Svelte and vanilla CSS.
- [spoken_transcript] First, head over to Svelte's beautiful website. They've got all the documentation and resources you need to get started.
- [spoken_transcript] I won't go over the setup process here, because it's much better if you follow the official docs and set it up yourself.
- [spoken_transcript] Once you're done, you should see a project structure like this. A source folder, a routes folder, and other files and folders that come with a typical Svelte project.
- [spoken_transcript] Instead of creating a style.css file like we did earlier, we'll create an app.css file inside the source folder.
- [spoken_transcript] Why app.css instead of style.css? It's just better, that's why.
- [on_screen_text] Tree Shaking Removes unused CSS automatically
- [on_screen_text] HMR Updates styles instantly without a full reload
- [on_screen_text] Code Splitting Loads only the required CSS per page
- [on_screen_text] Optimization Processes CSS with other assets for better performance
- [spoken_transcript] The app.css file will contain all the global styles we used earlier, except for the component-specific styles.
- [spoken_transcript] Speaking of components, let's start creating actual components. We'll create a folder called components, and inside it, we'll create a Header component.
- [spoken_transcript] Svelte files use the .svelte extension, but they work just like regular HTML files. You can use the same tags and syntax you're already familiar with from plain HTML.
- [spoken_transcript] Next, let's move all the header-related CSS into this Header.svelte file. This will help avoid any naming conflicts or cascading issues down the line.
- [spoken_transcript] Once that's done, open the page.svelte file, and include the Header component like this.
- [on_screen_text]

```
2 import Header from "../components/Header.svelte";
3 </script>
4 <Header />
```
- [spoken_transcript] You can think of this file as the index.html of your Svelte app. It's the entry point to your website. Any other pages will go inside the routes folder, just like this.

- [spoken_transcript] It's a good idea to install the Svelte extension for your IDE. This way, you won't have to manually import your components every time, and your formatting will look all clean and pretty.
- [spoken_transcript] Creating dynamic components in Svelte is almost identical to how we do it with web components, but even simpler.
- [spoken_transcript] The cool thing with Svelte is its built-in prop handling. You just export a variable, and that's it. No more doing this `getAttribute` thing manually.
- [on_screen_text]


```

2 export let heading;
3 export let details;
4 export let link;
5 export let linkText;
6 export let img;
      
```
- [spoken_transcript] Plus, it's reactive, meaning any changes automatically update the UI. But I'm getting ahead of myself here.
- [spoken_transcript] So to use the components, just do what we did earlier. Pass values manually or use an API if you're building a dynamic UI.
- [spoken_transcript] The final setup should look something like this. You have a source folder for all your components, and the `app.css` file for global styling. Your pages will be inside the routes folder, and any static assets like images go in the static folder.
- [spoken_transcript] Now here's the thing about Svelte. It's not just a runtime framework, it's a compiler. Unlike React or Vue, which use a virtual DOM, Svelte transforms your components into optimized vanilla code during build time.
- [spoken_transcript] This means you get the performance of a static site, but with the dynamic flexibility of an app.
- [spoken_transcript] If your project is mostly static with some interactive elements, like buttons, forms or modals, Svelte ensures only those parts stay dynamic. Everything else remains pure HTML and CSS.
- [spoken_transcript] So if you're building a mostly static website, React might be overkill. Svelte gives you the simplicity of plain HTML with modern interactivity, minus the extra fluff that can slow things down.

PHASE 13: Tailwind CSS Approach

VERBATIM STREAM:

- [on_screen_text] STEP 5 Create & Style your components
- [on_screen_text] v3 - Svelte + Tailwind
- [spoken_transcript] For large-scale, dynamic apps, use whatever framework you or your team are comfortable with.
- [spoken_transcript] But if you're looking for something simple, using a UI library like Tailwind can save you time.
- [spoken_transcript] Sure, it requires more effort upfront, but it also gives you more control.
- [spoken_transcript] Personally, I prefer writing my own custom CSS, but that's just me.

- [spoken_transcript] For the sake of comparison, let's see how you design the same website with Tailwind.
- [spoken_transcript] First, head over to the official Tailwind docs to check out the installation process. Luckily, Svelte makes it super easy to install Tailwind by default.
- [spoken_transcript] Pick the method that works best for you. And don't forget to install the Tailwind IntelliSense plugin for autocomplete and syntax highlighting.
- [spoken_transcript] Make sure to manually create the config file if you don't already have it. The plugin might not work properly without it.
- [spoken_transcript] Then, create the app.css just like we did earlier, but this time, don't forget to import Tailwind CSS.
- [on_screen_text]


```
1 @import "tailwindcss";
```
- [spoken_transcript] Now, let's move on to step two, creating our variables.
- [spoken_transcript] The naming will be a bit different, and we don't need to worry about spacing variables. Tailwind gives us all the margin and padding classes we could ever need.
- [spoken_transcript] Now let's set up our custom font classes. It'll look something like this.
- [on_screen_text]


```
17 /* Utilities */
18 @layer utilities {
19   .font-h1 {
20     font: var(--h1);
21   }
22   @apply my-2;
```
- [spoken_transcript] The only difference here is that we're using Tailwind's utility class for margins, rather than manually passing CSS variables like we did before.
- [spoken_transcript] In the base layer, we'll apply all the global styles needed for our components. And that's pretty much for the setup.
- [on_screen_text]


```
31 /* Global Styles */
32 @layer base {
33   html {
34     scroll-behavior: smooth;
35   }
```
- [spoken_transcript] Now let's dive into the components and move all the styles directly into the HTML markup like this.
- [on_screen_text]


```
6 <div class="mt-40 mb-20 text-center py-20 bg-a-70">
7   <h2 class="font-h1">{heading}</h2>
8   <p>{details}</p>
9 </div>
```
- [spoken_transcript] If you've never used Tailwind before, this might seem a little confusing or even ugly at first. And honestly, I get it.

- [spoken_transcript] But if you're already familiar with Tailwind, this might look a lot cleaner than what we were doing earlier. And that's pretty cool too.
- [spoken_transcript] But here's the thing. Clients will never care about what methods you use to build the website. All they care about is the final product.
- [spoken_transcript] So honestly, do what makes you happy.

PHASE 14: Conclusion

VERBATIM STREAM:

- [spoken_transcript] That said, here's what the final code looks like. I'll show both versions, and it's totally up to you which one you prefer to explore.
- [spoken_transcript] Personally, I always recommend starting with the vanilla solutions, before jumping into frameworks or libraries. You'll get a solid understanding of the fundamentals.
- [spoken_transcript] But anyway, back to the topic. The smart way of building websites. Design systems and components.
- [spoken_transcript] Whether you use vanilla solutions, frameworks, or no-code platforms, the basic principles will stay the same.
- [spoken_transcript] So first things first. Design your website. If design isn't your strong suit, hire someone, or check out some videos to get a better feel for it.
- [spoken_transcript] You can also find inspiration on platforms like Mobbin or the Figma community.
- [spoken_transcript] When designing, make sure to use components and variables for structuring and styling your content.
- [spoken_transcript] And if you're designing dynamic apps, this becomes a must.
- [spoken_transcript] Once you have a design, set up your variables and global styles.
- [spoken_transcript] With your basic styles in place, you can start creating your components using whatever framework or tool suits you best.
- [spoken_transcript] Then, it's all about how and when you use those components across your website.

TEACHABLES:

1. TYPE: procedure
 VERBATIM:
 STEP 0 Design first, build later
 STEP 1 Define your components
 STEP 2 Setup your CSS variables
 STEP 3 Create global styles
 STEP 4 Create utility styles
 STEP 5 Create & Style your components
2. TYPE: example
 VERBATIM:
 /* Reset */

- {
margin: 0;
padding: 0;
}
box-sizing: border-box;
- 3.
- 4. TYPE: example
VERBATIM:
/* Variables */
:root {
--color10: hsl(31, 80%, 10%);
--color30: hsl(31, 80%, 30%);
--color90: hsl(31, 80%, 90%);
--color100: hsl(31, 80%, 100%);
--accent: hsl(31, 80%, 70%);
--ff: "Noto Sans JP", serif;
--ff2: "Fira Sans", serif;
--p: 1rem/1.5em var(--ff);
--h1: 600 3rem/1.2em var(--ff2);
--h2: 600 2rem/1.2em var(--ff2);
--spacing1: 4px;
--spacing2: 8px;
--spacing3: 16px;
--spacing4: 20px;
--spacing5: 40px;
--spacing6: 80px;
--spacing7: 160px;
}
5. TYPE: example
VERBATIM:
/* Global Styles */
html {
scroll-behavior: smooth;
}
body {
text-wrap: balance;
font: var(--p);
background: var(--color90);
color: var(--color10);
}
h1 {
font: var(--h1);
}
h2 {

```

font: var(--h2);
}
.btn,
button {
all: unset;
display: inline-block;
margin: var(--spacing2) 0;
background: var(--color100);
padding: var(--spacing2) var(--spacing3);
color: var(--color10);
border-radius: var(--spacing2);
border-bottom: solid var(--spacing1) var(--accent);
font: var(--p);
cursor: pointer;
}

```

6. TYPE: example

VERBATIM:

```

/* Utility Classes */
section {
width: 90%;
max-width: 1080px;
margin: 0 auto;
}
.center {
text-align: center;
}
.hidden {
display: none;
}
.flex {
display: flex;
align-items: center;
justify-content: center;
gap: var(--spacing4);
flex-wrap: wrap;
}

```

7. TYPE: example

VERBATIM:

```

// header
class THeader extends HTMLElement {
connectedCallback() {
this.innerHTML = `
<header>


```

```


<nav>
  <a href="#">Services</a>
  <a href="#">Our Work</a>
  <a href="#">Pricing</a>
  <a href="#">About</a>
  <a class="btn" href="#">Hire us</a>
</nav>
</header>
`;
}
}
customElements.define("t-header", THeader);

```

8. TYPE: example

VERBATIM:

```

// section
class TSection extends HTMLElement {
  connectedCallback() {
    const heading = this.getAttribute("heading");
    const details = this.getAttribute("details");
    const link = this.getAttribute("link");
    const linkText = this.getAttribute("linkText");
    const img = this.getAttribute("img");
    this.innerHTML = `
    <section class="flex margin6">
      <div>
        <h2>${heading}</h2>
        <p>${details}</p>
        <a class="btn" href="${link}">${linkText}</a>
      </div>
      
    </section>
    `;
  }
}
customElements.define("t-section", TSection);

```

9. TYPE: example

VERBATIM:

```

<t-section
  heading="We Build Websites"
  details="Ever dreamed of a website that loads in under a minute? We make that
  happen-sometimes even faster."

```

```
link="#"
linkText="Explore the magic"
img="img/building websites.svg"
</t-section>
```

10. TYPE: example

VERBATIM:

```
class TMainSection extends HTMLElement {
  connectedCallback() {
    const heading = this.getAttribute("heading");
    const details = this.getAttribute("details");
    this.innerHTML = `
    <section class="center padding6">
    <h2>${heading}</h2>
    <p>${details}</p>
    </section>
    `;
  }
}

customElements.define("t-main-section", TMainSection);
```

11. TYPE: example

VERBATIM:

```
/* Tailwind Utilities /
@layer utilities {
  .font-h1 {
    font: var(--h1);
    @apply my-2;
  }
}

/ Global Styles */
@layer base {
  html {
    scroll-behavior: smooth;
  }
}
```